

Family Module

Integration & Implementation Guide

Version: 1.0

Date: August 29, 2026

Backend: Django + Django REST Framework

Module: families

1. Purpose

The Family module allows users to create and manage family groups, associate patients with those families, invite additional family members, and allow pending patients to claim their own accounts.

The module supports two major invitation flows:

1. **Family Member Invitation**
2. **Patient Account Claim Invitation**

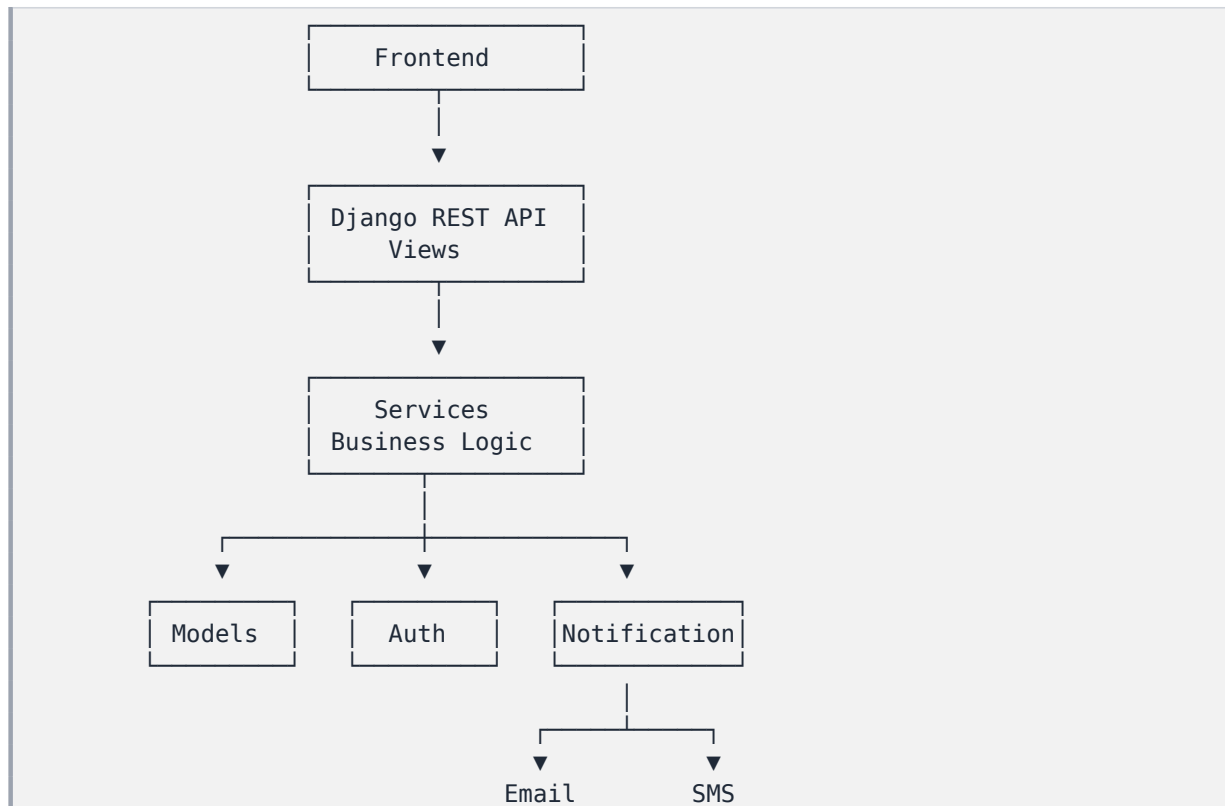
Both flows use:

- Secure invitation tokens
- Contact verification
- One-time passwords (OTP)
- Invitation expiration
- Atomic database transactions
- Service-layer business logic
- Existing-account detection
- Pending-account activation

The objective is to keep the API views thin while putting all business rules and state transitions inside the service layer.

2. Architecture

The recommended architecture is:



Responsibility by layer

Views

Views should:

- Authenticate requests
- Validate serializers
- Call services
- Convert service errors into HTTP responses
- Return serialized responses

Views should **not** contain complex business logic.

Serializers

Serializers should:

- Validate request data
- Validate field formats

- Validate passwords
- Validate email/phone formats
- Serialize API responses

Services

Services should handle:

- Invitation creation
- Token generation
- Token hashing
- OTP generation
- OTP verification
- Invitation expiration
- Account creation
- Account activation
- Family membership creation
- Patient claiming
- Invitation consumption
- Transaction boundaries

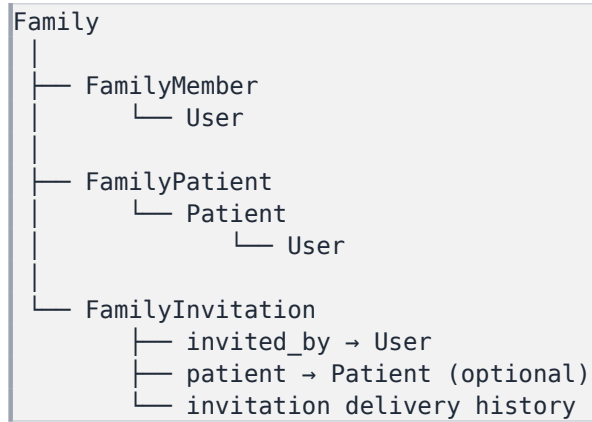
Models

Models should represent:

- Families
- Family members
- Family patients
- Invitations
- Delivery attempts

3. Core Domain Objects

The module revolves around five primary objects.



4. Family

A **Family** represents a group of users and patients.

Typical fields:

```
id
name
created_by
created_at
updated_at
```

When a family is created, the creator automatically becomes:

```
OWNER
```

The owner receives the initial management permissions.

5. FamilyMember

FamilyMember represents the relationship between a User and a Family.

Typical fields:

```
family
user
role

can_view_patient_records
can_manage_appointments
can_manage_medications
can_manage_family_members
can_manage_family_patients

created_at
updated_at
```

The owner is initially granted all family management permissions.

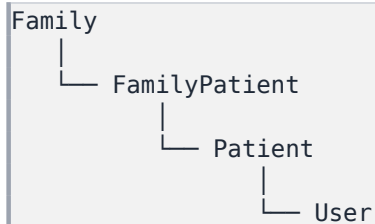
Example:

```
Family
├── Esrael
│   └── role = OWNER
├── Sara
│   └── role = MEMBER
└── Daniel
    └── role = MEMBER
```

6. FamilyPatient

FamilyPatient connects an existing **Patient** to a **Family**.

It should not duplicate the actual patient record.



This distinction is important.

A patient belongs to the healthcare system independently of the family relationship.

The family relationship is represented by **FamilyPatient**.

7. Pending Patient Accounts

When a family creates a patient who does not yet have an account:

```
User  
account_status = pending  
password = unusable
```



```
Patient
```



```
FamilyPatient
```

The patient can later claim the account.

The claim operation must **not create another Patient**.

Instead:

```
Pending User
```



```
Existing Patient
```



```
Activate User
```

8. FamilyInvitation

FamilyInvitation represents an invitation that can be accepted by a recipient.

There are two types:

```
class InvitationType(models.TextChoices):  
    MEMBER = "member", "Family Member"  
    PATIENT_CLAIM = "patient_claim", "Patient Claim"
```

Typical statuses:

```
class Status(models.TextChoices):  
    PENDING = "pending", "Pending"  
    ACCEPTED = "accepted", "Accepted"  
    CANCELLED = "cancelled", "Cancelled"  
    EXPIRED = "expired", "Expired"
```

Typical fields:

```
family  
invited_by  
  
invitation_type  
status  
  
name  
email  
phone_number  
  
role  
  
patient  
  
token_hash  
expires_at  
  
otp_hash  
otp_expires_at  
otp_attempts  
contact_verified_at  
  
accepted_at  
  
created_at  
updated_at
```

9. Invitation Security

The raw invitation token must never be stored in the database.

The service generates:

```
raw_token = secrets.token_urlsafe(32)
```

Then hashes it:

```
token_hash = hashlib.sha256(  
    raw_token.encode("utf-8")  
) .hexdigest()
```

Only:

```
token_hash
```

is stored.

The raw token is sent to the recipient.

Therefore:

```
Database  
└─ SHA-256(token)  
  
Recipient  
└─ raw token
```

If the database is compromised, the raw invitation URLs are not directly exposed.

10. Invitation Expiration

The default invitation lifetime is:

```
INVITATION_EXPIRY_DAYS = 7
```

When an invitation is created:

```
expires_at = (  
    timezone.now()  
    + timedelta(days=7)  
)
```

Every invitation lookup must verify expiration.

If expired:

```
PENDING → EXPIRED
```

The invitation must then be rejected.

11. Invitation Token Flow

The frontend receives an invitation URL such as:

```
https://app.example.com/family/invitation/<token>
```

The frontend uses the token to request invitation details.

```
GET /api/families/invitations/<token>/
```

The backend:

3. Hashes token
4. Finds invitation
5. Checks expiration
6. Checks status
7. Returns safe invitation information

The raw token should never be stored permanently by the backend.

12. Family Creation Flow

Endpoint:

```
POST /api/families/
```

Request:

```
{  
  "name": "Nathan Family"  
}
```

The view calls:

```
family = create_family(  
    user=request.user,  
    name=serializer.validated_data["name"],  
)
```

The service creates:

```
Family  
+  
FamilyMember
```

The creator becomes:

```
OWNER
```

with management permissions enabled.

13. Creating a Family Patient

Endpoint:

```
POST /api/families/<family_id>/patients/
```

Example request:

```
{
  "first_name": "John",
  "last_name": "Nathan",
  "email": "john@example.com",
  "phone_number": "+2519000000000",
  "gender": "male",
  "blood_group": "O+",
  "relationship": "son",
  "is_primary": false
}
```

The service performs:

```
Find User
├── Existing → reuse
└── Not found
    │
    ▼
  Create User
  status=pending
```

Then:

```
User
|
▼
Patient
|
▼
FamilyPatient
```

Duplicate family relationships are rejected.

14. Family Member Invitation

Endpoint:

```
POST /api/families/<family_id>/invitations/member/
```

Example:

```
{
  "name": "Sarah Nathan",
  "email": "sarah@example.com",
  "role": "member"
}
```

The service:

8. Finds existing account if one exists.
9. Checks whether the person is already a family member.
10. Cancels an existing pending invitation for the same contact.
11. Generates a secure token.
12. Hashes the token.
13. Creates the invitation.
14. Returns the invitation and raw token internally.
15. Passes the raw token to the notification layer.

The raw token should **not** be included in the normal API response.

15. Sending the Invitation

The service should eventually trigger a notification task.

Example:

```
send_family_invitation.delay(  
    invitation.id,  
    raw_token,  
)
```

The notification service constructs:

```
https://app.example.com/family/invitations/<raw_token>
```

and sends it through email or SMS.

The service layer should not contain email/SMS implementation details.

16. Invitation Contact Verification

Once the recipient opens the invitation:

```
GET /api/families/invitations/<token>/
```

The frontend can show:

```
You have been invited to join:  
  
Nathan Family  
  
Role:  
Member  
  
Verification:  
Email
```

The frontend then requests verification:

```
POST /api/families/invitations/<token>/verify/request/
```

The service:

16. Resolves invitation.
17. Checks status.
18. Checks expiration.
19. Determines email or SMS.
20. Generates a six-digit OTP.
21. Hashes the OTP.
22. Stores OTP hash.
23. Sets expiration.
24. Resets attempts.
25. Creates **InvitationDelivery**.
26. Sends OTP through notification infrastructure.

17. OTP Storage

Never store:

```
123456
```

Store:

```
SHA256(123456)
```

Example:

```
otp_hash = hashlib.sha256(  
    otp.encode("utf-8")  
) .hexdigest()
```

OTP lifetime:

```
OTP_EXPIRY_MINUTES = 10
```

Maximum attempts:

```
OTP_MAX_ATTEMPTS = 5
```

18. OTP Verification

Endpoint:

```
POST /api/families/invitations/<token>/verify/
```

Request:

```
{  
  "otp": "123456"  
}
```

The service:

27. Resolves invitation.
28. Checks expiration.
29. Checks whether an OTP exists.
30. Checks attempt count.
31. Hashes supplied OTP.
32. Compares hashes.
33. Increments attempts on failure.
34. Marks contact verified on success.

Successful state:

```
contact_verified_at != NULL
```

The invitation is still:

```
PENDING
```

OTP verification does **not** accept the invitation.

19. Existing User Acceptance

An existing user follows:

```
Invitation
  ↓
OTP verification
  ↓
Login
  ↓
Accept invitation
```

Endpoint:

```
POST /api/families/invitations/<token>/accept/
```

Authentication is required.

The service verifies:

```
Invitation valid
  +
Invitation type = MEMBER
  +
Contact verified
  +
Authenticated user matches invitation contact
  +
User not already member
```

Then:

```
FamilyMember created
```

and:

```
Invitation
PENDING → ACCEPTED
```

20. New User Registration

If the invited person does not have an account:

```
Invitation
  ↓
OTP verification
  ↓
Registration
  ↓
User creation
  ↓
FamilyMember creation
  ↓
Invitation accepted
```

Endpoint:

```
POST /api/families/invitations/<token>/register/
```

Example:

```
{
  "username": "sarah",
  "password": "SecurePassword123!",
  "first_name": "Sarah",
  "last_name": "Nathan"
}
```

The service:

35. Validates invitation.
36. Requires contact verification.
37. Checks account collisions.
38. Creates User.
39. Sets password.
40. Sets account status to **active**.
41. Creates FamilyMember.
42. Marks invitation accepted.

All operations happen inside one transaction.

21. Patient Claim Flow

Patient claiming is different from family-member registration.

The patient already exists:

```
User
|
▼
Patient
|
▼
FamilyPatient
```

The user is simply:

```
account_status = pending
```

The family creates a patient claim invitation.

Endpoint:

```
POST /api/families/<family_id>/patients/<patient_id>/claim/
```

The service:

43. Checks family access.
44. Checks patient belongs to family.
45. Checks account is not already active.
46. Cancels old pending claim invitations.
47. Creates a new invitation.
48. Generates secure token.

22. Completing Patient Claim

Endpoint:

```
POST /api/families/invitations/<token>/claim/
```

Example:

```
{
  "username": "john",
  "password": "SecurePassword123!"
}
```

The service verifies:

```
Invitation valid
+
Type = PATIENT_CLAIM
+
Contact verified
+
Patient exists
+
User exists
+
User is pending
```

Then:

```
User
account_status = pending
|
▼
account_status = active
```

The existing Patient remains unchanged.

The existing FamilyPatient relationship remains unchanged.

23. Patient Claim Data Integrity

This is a critical rule.

Do NOT do:

```
Patient.objects.create(...)
```

during patient claiming.

Instead:

```
patient = invitation.patient  
user = patient.user
```

Then activate the existing user.

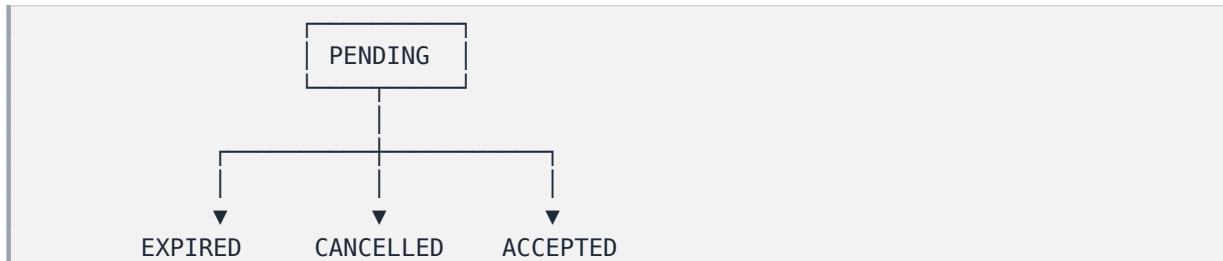
The result is:

```
One User  
One Patient  
One FamilyPatient
```

rather than duplicate records.

24. Invitation State Machine

The intended lifecycle is:



OTP verification does not change the invitation status.

Instead:

```
PENDING
+
contact_verified_at
```

means:

```
Invitation is verified and ready for completion.
```

25. Delivery History

Every OTP/invitation delivery should be recorded in:

```
InvitationDelivery
```

Example:

```
Invitation
├── Email → delivered
├── Email → failed
└── SMS   → delivered
```

Useful fields:

```
invitation
channel
destination
status
created_at
```

This provides an audit trail for delivery attempts.

26. API Endpoint Summary

Recommended URL structure:

```
/api/families/
```

Families

```
GET    /api/families/
POST   /api/families/

GET    /api/families/<family_id>/
PATCH /api/families/<family_id>/
```

Members

```
GET /api/families/<family_id>/members/
```

Patients

```
GET /api/families/<family_id>/patients/
POST /api/families/<family_id>/patients/

GET /api/families/<family_id>/patients/<patient_id>/
```

Member invitations

```
POST /api/families/<family_id>/invitations/member/
```

Patient claim invitations

```
POST /api/families/<family_id>/patients/<patient_id>/claim/
```

Public invitation

```
GET /api/families/invitations/<token>/
```

Contact verification

```
POST /api/families/invitations/<token>/verify/request/
POST /api/families/invitations/<token>/verify/
```

Existing-user acceptance

```
POST /api/families/invitations/<token>/accept/
```

New-user registration

```
POST /api/families/invitations/<token>/register/
```

Patient claim completion

```
POST /api/families/invitations/<token>/claim/
```

Delivery history

```
GET /api/families/<family_id>/invitations/<invitation_id>/deliveries/
```

27. Permission Model

Recommended authorization:

Operation	Required Permission
View family	Family member
Update family	Family member / manager
View members	Family member
Invite member	Manage family members
View patients	Family member
Create family patient	Manage family patients
Create patient claim invitation	Manage family patients
Accept invitation	Authenticated user
Register invitation	Public + verified invitation
Claim patient	Public + verified invitation
View delivery history	Authenticated + family access

The public invitation endpoints should never expose sensitive patient medical information.

28. Medical Data Isolation

The Family module should not automatically expose:

```
Medical records  
Lab results  
Prescriptions  
Medical documents  
Clinical notes  
Appointments  
Vitals
```

Simply being a family member should not automatically mean unlimited access.

Family membership should determine the relationship and permissions.

Clinical modules should perform their own authorization checks.

For example:

```
FamilyMember  
|  
▼  
can_view_patient_records  
|  
▼  
Medical Records Permission  
|  
▼  
Medical Records API
```

This keeps healthcare data authorization centralized and safer.

29. Frontend Integration

The frontend invitation page can use this flow.

Step 1

User opens:

```
/family/invitations/<token>
```

Step 2

Frontend calls:

```
GET /api/families/invitations/<token>/
```

Step 3

Display:

```
Family:
Nathan Family

Invitation:
Family Member

Role:
Member

Contact:
s***@example.com
```

Step 4

User selects:

```
Send verification code
```

Step 5

Frontend calls:

```
POST /api/families/invitations/<token>/verify/request/
```

Step 6

User enters OTP.

```
POST /api/families/invitations/<token>/verify/
```

Step 7

Frontend receives:

```
{  
  "verified": true  
}
```

Step 8

Determine account state.

If user already has an account:

```
Login  
↓  
Accept Invitation
```

If no account:

```
Registration  
↓  
Complete Invitation
```

30. Recommended Frontend State

The invitation page should maintain:

```
type InvitationState =  
  | "loading"  
  | "invalid"  
  | "pending"  
  | "otp_required"  
  | "verified"  
  | "completed";
```

Example:

```
loading  
  ↓  
pending  
  ↓  
otp_required  
  ↓  
verified  
  ↓  
completed
```

Errors should include:

```
expired  
cancelled  
already accepted  
invalid token  
OTP expired  
too many attempts  
account already exists
```

31. Notification Integration

The notification layer should be independent from invitation business logic.

Recommended structure:

```
families/  
  services.py  
  notifications.py  
  tasks.py
```

For example:

```
def send_family_invitation(  
    *,  
    invitation,  
    raw_token,  
):  
    ...
```

and:

```
def send_invitation_otp(  
    *,  
    invitation,  
    otp,  
):  
    ...
```

Celery can execute these asynchronously:

```
send_family_invitation.delay(  
    invitation.id,  
    raw_token,  
)
```

and:

```
send_invitation_otp.delay(  
    invitation.id,  
    otp,  
)
```

32. Important Notification Rule

Never put the raw invitation token into:

```
database logs  
application logs  
analytics  
error messages
```

Never log:

```
print(raw_token)
```

Never log:

```
logger.info(  
    "OTP=%s",  
    otp,  
)
```

Production logs should contain identifiers such as:

```
invitation_id  
delivery_id  
channel  
status
```

but not secrets.

33. Rate Limiting

The OTP request endpoint should eventually be rate-limited.

Recommended protections:

```
Per invitation:  
    maximum OTP requests per time window  
  
Per IP:  
    maximum verification requests  
  
Per contact:  
    maximum OTP requests  
  
Per invitation:  
    maximum failed OTP attempts
```

For example:

```
OTP validity:  
10 minutes  
  
Verification attempts:  
5  
  
OTP resend:  
60 seconds minimum
```

The exact values can be adjusted according to operational requirements.

34. Transaction Safety

Account creation and invitation acceptance must be atomic.

Use:

```
@transaction.atomic
```

For example:

```
Create User
|
Create FamilyMember
|
Accept Invitation
```

If any step fails:

```
ROLLBACK
```

This prevents states such as:

```
User created
FamilyMember missing
Invitation still pending
```

35. Concurrency Protection

For production use, invitation completion should also consider concurrent requests.

Two requests could theoretically attempt:

```
POST /register/  
POST /register/
```

at nearly the same time.

The database should enforce uniqueness where appropriate.

Recommended constraints include:

```
FamilyMember(family, user)  
FamilyPatient(family, patient)  
Invitation token hash
```

and potentially appropriate uniqueness constraints for active invitations.

The service layer should still perform logical checks.

Database constraints provide the final protection.

36. Serializer Responsibilities

Recommended serializers:

```
FamilySerializer
FamilyMemberSerializer
FamilyPatientSerializer
FamilyPatientDetailSerializer
FamilyInvitationSerializer
InvitationDeliverySerializer
CreateFamilyMemberInvitationSerializer
CreateFamilyPatientSerializer
CreatePatientClaimInvitationSerializer
AcceptInvitationSerializer
VerifyInvitationOTPSerializer
CompleteInvitationRegistrationSerializer
```

The registration serializer should validate:

```
username
password
password confirmation
first name
last name
```

The OTP serializer should validate:

```
otp
```

with:

```
exactly 6 digits
```

37. Service API

The final service API is:

```
create_family()
create_family_patient()
invite_family_member()
create_patient_claim_invitation()
get_invitation_by_token()
request_invitation_contact_verification()
verify_invitation_otp()
accept_invitation()
complete_invitation_registration()
complete_patient_claim()
```

Helper functions:

```
generate_invitation_token()
hash_invitation_token()
generate_otp()
hash_otp()
_require_verified_invitation()
```

38. Error Handling

Services should raise:

```
ValueError(...)
```

for expected business errors.

Views convert them to:

```
400 Bad Request
```

Example:

```
try:
    member = accept_invitation(
        token=token,
        user=request.user,
    )
except ValueError as exc:
    return Response(
        {"detail": str(exc)},
        status=status.HTTP_400_BAD_REQUEST,
    )
```

For invalid invitation tokens, the lookup endpoint may return:

```
404 Not Found
```

This avoids exposing whether a particular token exists.

39. Testing Strategy

The Family module should have tests for every state transition.

Family creation

Test:

```
family created  
owner created  
owner permissions assigned
```

Patient creation

Test:

```
new user created  
pending status assigned  
patient created  
family patient created
```

Existing user:

```
existing user reused  
patient reused if present
```

Duplicate:

```
same family + patient rejected
```

40. Invitation Tests

Test:

```
invitation creation  
token hashing  
token lookup  
expired token  
cancelled invitation  
accepted invitation  
duplicate invitation
```

Verify that:

```
raw token is never stored
```

41. OTP Tests

Test:

```
OTP generated
OTP hashed
OTP expires
wrong OTP
correct OTP
maximum attempts
expired OTP
OTP reset
contact_verified_at
```

Important:

```
OTP verification ≠ invitation acceptance
```

42. Existing User Tests

Test:

```
verified existing user accepts  
unverified user rejected  
wrong user rejected  
already member rejected  
wrong invitation type rejected  
expired invitation rejected
```

43. Registration Tests

Test:

```
verified invitation creates user  
password is usable  
account becomes active  
FamilyMember created  
invitation becomes accepted  
duplicate email rejected  
duplicate username rejected
```

Also test rollback:

```
If FamilyMember creation fails:  
  User creation rolls back
```

44. Patient Claim Tests

Test:

```
pending patient can claim  
active patient cannot claim  
unverified invitation rejected  
wrong invitation type rejected  
existing Patient retained  
existing FamilyPatient retained  
User becomes active  
password assigned
```

Most importantly:

```
assert Patient.objects.count() == original_count
```

after claiming.

45. Production Checklist

Before considering the module production-ready:

Security

- [] Invitation tokens hashed
- [] OTPs hashed
- [] OTP expiration implemented
- [] OTP attempt limit implemented
- [] Invitation expiration implemented
- [] Raw tokens not logged
- [] Raw OTPs not logged
- [] Public endpoints expose minimal data
- [] Contact information masked
- [] Authentication required where appropriate
- [] Family permissions enforced

Data integrity

- [] FamilyMember unique constraint
- [] FamilyPatient unique constraint
- [] Invitation lifecycle enforced
- [] Patient claim does not create duplicate Patient
- [] Transactions used for state-changing flows

Notifications

- [] Email delivery implemented
- [] SMS delivery implemented where required
- [] Delivery history recorded
- [] Failed delivery recorded
- [] Retry mechanism implemented
- [] Celery/background processing configured

API

- [] Serializers validated
- [] URLs registered
- [] Permissions configured
- [] Error responses standardized
- [] Pagination added where needed

Testing

- [] Unit tests
- [] Service tests
- [] API tests
- [] Permission tests

- [] OTP tests
- [] Invitation expiry tests
- [] Concurrency tests

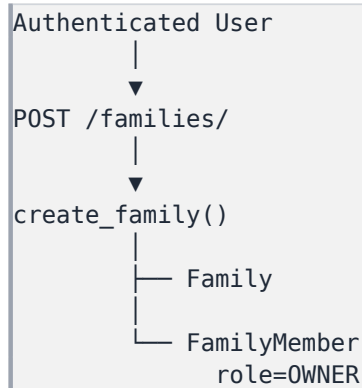
46. Recommended Final Project Structure

A clean implementation should look approximately like:

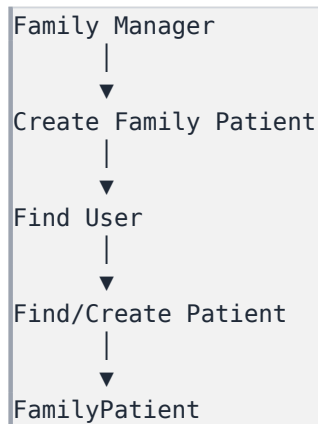
```
families/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── permissions.py
├── serializers.py
├── services.py
├── notifications.py
├── tasks.py
├── urls.py
├── views.py
├── tests/
│   ├── __init__.py
│   ├── test_models.py
│   ├── test_services.py
│   ├── test_invitations.py
│   ├── test_patient_claim.py
│   ├── test_permissions.py
│   └── test_api.py
└── migrations/
```

47. Final End-to-End Flows

Flow A — Create Family



Flow B — Add Existing Patient



Flow C — Invite Existing User





```
graph TD; A[OTP Verification] --> B[Login]; B --> C[Accept Invitation]; C --> D[FamilyMember];
```

OTP Verification

▼

Login

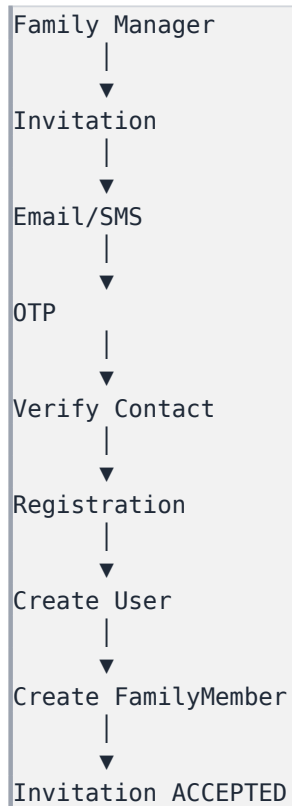
▼

Accept Invitation

▼

FamilyMember

Flow D — Invite New User



```
graph TD; A[Family Manager] --> B[Invitation]; B --> C[Email/SMS]; C --> D[OTP]; D --> E[Verify Contact]; E --> F[Registration]; F --> G[Create User]; G --> H[Create FamilyMember]; H --> I[Invitation ACCEPTED];
```

Family Manager

▼

Invitation

▼

Email/SMS

▼

OTP

▼

Verify Contact

▼

Registration

▼

Create User

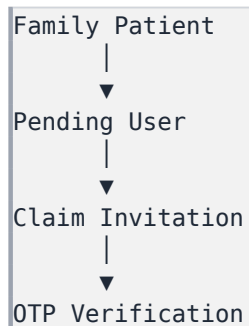
▼

Create FamilyMember

▼

Invitation ACCEPTED

Flow E — Patient Account Claim



```
graph TD; A[Family Patient] --> B[Pending User]; B --> C[Claim Invitation]; C --> D[OTP Verification];
```

Family Patient

▼

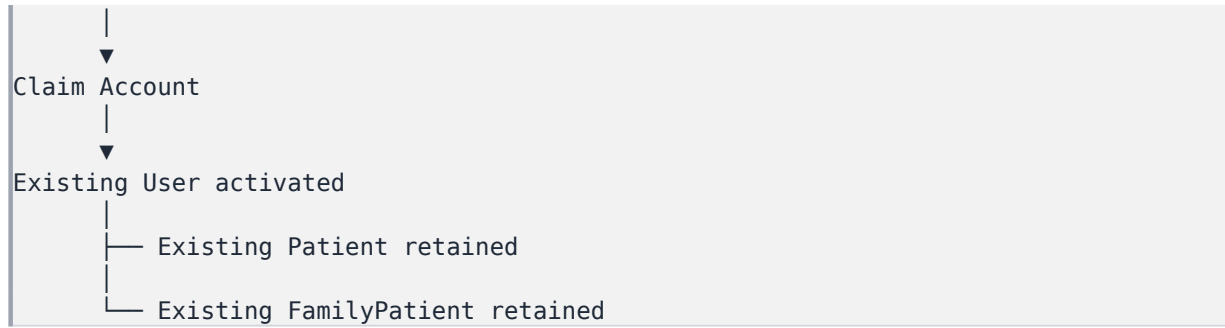
Pending User

▼

Claim Invitation

▼

OTP Verification



48. Key Design Principles

The Family module should follow these rules throughout the application:

Rule 1

Views coordinate; services decide.

Rule 2

Never store raw invitation tokens.

Rule 3

Never store raw OTPs.

Rule 4

OTP verification and invitation acceptance are separate operations.

Rule 5

A patient claim activates an existing account; it does not create another patient.

Rule 6

Invitation completion must be atomic.

Rule 7

Family membership does not automatically grant access to all medical information.

Rule 8

Notification delivery belongs outside the core invitation business logic.

Rule 9

Database constraints should reinforce service-level validation.

Rule 10

Public invitation endpoints should expose the minimum information required to complete the flow.

49. Implementation Order

For implementation, the recommended order is:

```
1. Models
   ↓
2. Migrations
   ↓
3. Serializers
   ↓
4. Services
   ↓
5. Permissions
   ↓
6. Views
   ↓
7. URLs
   ↓
8. Admin
   ↓
9. Notification services
   ↓
10. Celery tasks
   ↓
11. Tests
   ↓
12. Frontend invitation UI
```

This keeps dependencies clear and makes debugging significantly easier.

50. Conclusion

The Family module provides a controlled relationship layer between users, families, and patients while keeping account creation and patient claiming separate from ordinary family membership.

The most important architectural distinction is:

```
Family Member
    =
User joining a Family

Patient Claim
    =
Existing Patient/User account becoming active
```

These should never be treated as the same operation.

With the service-layer implementation, invitation token hashing, OTP verification, atomic account creation, patient-account preservation, and permission checks, the module has a strong foundation for production integration.

The remaining production integrations are primarily:

```
Email/SMS notification provider
Celery/background tasks
Rate limiting
Database constraints
Automated tests
Frontend invitation screens
```

Once those are connected, the Family module can serve as the stable relationship and invitation layer for the rest of the healthcare platform.